

Codebook of UltraGrammer

ahbiee

Contents

1 你是否有...	1
1.1 嘗試...	1
1.2 檢查...	1
2 資料結構: 前綴和	1
2.1 BIT (Fenwick Tree)	1
2.2 Segment Tree 線段樹	1
3 數論: 尤拉公式, 數論分塊, Sprague-Grundy 函數, 排容原理	2
3.1 快速幂	2
3.2 模逆元與費馬小定理	2
3.3 質數檢查	2
3.4 大數乘法	2
4 字串	3
4.1 KMP 字串匹配	3
4.2 Z-Algorithm 字串匹配	3
4.3 Manacher 迴文判斷	3
4.4 Trie 字典樹	3
4.5 AC 自動機	4
5 圖論: 著色問題, 帶權二分圖最大匹配 (KM 演算法), 最大流, biconnected component 與 articulation point 找 dfn 與 low	4
5.1 BFS 與拓撲排序	4
5.2 DFS	5
5.3 匈牙利演算法	5
5.4 Dijkstra 單源最短路徑	5
5.5 SPFA 單源最短路徑	5
5.6 Floyd-Warshall 多源最短路徑	6
6 幾何	6
6.1 點與向量基礎 (Point & Vector)	6
6.2 判斷線段相交 (Segment Intersection)	6
6.3 點與線段關係 (Point & Line)	6
6.4 任意多邊形面積	6
6.5 凸包 Convex Hull	7
7 樹論	7
7.1 Disjoint Set	7
7.2 Kruskal 最小生成樹	7
8 DP: LIS, LDS, 分錢幣問題, Kadane's algo 找最大子序列問題	7
8.1 01 Knapsack 背包	7
8.2 LCS 最長共同子序列 (Sequence)	8
8.3 區間 DP 概念 (最長迴文子序列)	8
8.4 跳格子	8
9 Greedy: 二分搜	9
9.1 一維區間全覆蓋	9
9.2 間格跳躍	9
9.3 最大不重疊區間	9
9.4 最小覆蓋區間	9
10 MISC: Random Select(QuickSort), 雙指針, 滑動窗口, C++ algorithm 庫	9
10.1 模板	9

1 你是否有...

1.1 嘗試...

- 再讀一遍題目
- 懷疑你認為「理所當然」的直覺?
- 試著寫下直覺的規律? (有助於觀察)
- 換個方向思考?
- 暴力解?
- 貪心解?
- 動態規劃 (DP)?
- 狀態建模, 把問題轉化成圖論?
- 只在奇數 index 反轉狀態?

- 用乘法取代除法? ($/2 = *2^{-1}$, 模逆元)

1.2 檢查...

- 確定沒漏看敘述?
- 確認數值範圍? (溢位, 精度位數, 陣列大小...)
- 不同的陣列/字串使用同個 index 導致超出範圍?
- 模板抄錯?
- 多筆資料之間初始化?
- 題目額外的敘述, 暗示?
- 未定義行為? (陣列越界, 未初始化)
- overflow? (int -> long long)
- 非 void function 忘記 return?
- 浮點數誤差? (極小的 eps, 是否需要 long double)
- 隱轉換? (int*int 爆了後才存到 long long)
- signed 與 unsigned int 之間的 compare?

2 資料結構: 前綴和

2.1 BIT (Fenwick Tree)

用法: 支援單點修改與區間求和。必須是 1-based index。

時間複雜度: $\mathcal{O}(\log n)$

```

1 struct BIT {
2     int n;
3     vector<long long> tree; // 使用 long long 避免區間總和溢位
4
5     void init(int sz){
6         n = sz;
7         tree.assign(sz+1, 0);
8     }
9
10    // lowbit: 取得二進位中最右邊的 1
11    inline int lowbit(int x) {
12        return x & (-x);
13    }
14
15    // 單點修改: 在位置 x 加上了 val
16    void add(int x, long long val) {
17        for (; x <= n; x += lowbit(x)) {
18            tree[x] += val;
19        }
20    }
21
22    // 前綴查詢: 計算 [1, x] 的總和
23    long long query(int x) {
24        long long sum = 0;
25        for (; x > 0; x -= lowbit(x)) {
26            sum += tree[x];
27        }
28        return sum;
29    }
30
31    // 區間查詢: 計算 [l, r] 的總和 (前綴和相減概念)
32    long long query(int l, int r) {
33        return query(r) - query(l - 1);
34    }
35 };

```

2.2 Segment Tree 線段樹

用法: 給定 n 筆資料查詢與 m 次範圍更新, 解決多次範圍更新的問題, 找區間和或最大/最小值。必須是 1-based index。

時間複雜度: 建樹 $\mathcal{O}(N)$ 查詢/更新 $\mathcal{O}(\log N)$

```

1 using ll = long long;
2
3 vector<ll> arr, sum, add; // 宣告全域, arr 是原始數據, sum 是範圍累加和, add
  是 lazy tag 儲存值
4
5 void up(int i){ // 往上加回去
6     sum[i] = sum[i*2] + sum[i*2+1];
7 }
8
9 void lazy(int i, ll v, int n){ // 懶標記 (lazy tag), 暫存當前覆蓋範圍
10    sum[i] += v*n;
11    add[i] += v;
12 }
13
14 void down(int i, int ln, int rn){ // 往下分發 lazy tag
15     if(add[i] != 0){
16         lazy(i*2, add[i], ln);
17         lazy(i*2+1, add[i], rn);
18         add[i] = 0;
19     }
20 }
21
22 void build(int l, int r, int i){ // 遞迴式初始化 (init)
23     if(l == r) sum[i] = arr[l]; // 如果區間只剩一個數值, 就直接賦值
24     else{
25         int mid = l + (r-l)/2; // 找中點

```

```

26         build(l, mid, i*2); // 左半邊 build
27         build(mid+1, r, i*2+1); // 右半邊 build
28         up(i); // 自己 = 左 + 右 區間和
29     }
30     add[i] = 0; // 初始化所有 add[i] = 0
31 }
32
33 void update(int jobl, int jobr, ll jobv, int l, int r, int i){ // 更新區間數
    值 (jobl ~ jobr 加上 jobv) 用 l, r, i 判斷範圍
34     if(jobl <= l && r <= jobr) lazy(i, jobv, r-l+1); // 如果範圍被包的話就直接
    lazy
35     else{
36         int mid = l + (r-l)/2;
37         down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
38         if(jobl <= mid) update(jobl, jobr, jobv, l, mid, i*2); // 判斷左右區
        段是否需要更新
39         if(jobr > mid) update(jobl, jobr, jobv, mid+1, r, i*2+1);
40         up(i); // 最後往上加總回去更新 sum
41     }
42 }
43
44 ll query(int jobl, int jobr, int l, int r, int i){
45     if(jobl <= l && r <= jobr) return sum[i]; // 如果包範圍就直接 return 回去
46
47     int mid = l + (r-l)/2;
48     down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
49     ll total = 0;
50     if(jobl <= mid) total += query(jobl, jobr, l, mid, i*2);
51     if(jobr > mid) total += query(jobl, jobr, mid+1, r, i*2+1);
52     return total; // 因為只是 query，沒有修改，所以不用 up 更新回去
53 }
54
55 int main() {
56     int n, m;
57     cin >> n >> m;
58     arr.assign(n+1, 0);
59     sum.assign(4*(n+1), 0); // 大小必須開到 4 倍 n+1
60     add.assign(4*(n+1), 0);
61
62     for(int i=1; i<=n; ++i) cin >> arr[i];
63     build(1, n, 1);
64     int op;
65     ll x, y, k;
66     while(m--){
67         cin >> op;
68         if(op == 1){
69             cin >> x >> y >> k;
70             update(x, y, k, 1, n, 1);
71         }
72         else{
73             cin >> x >> y;
74             cout << query(x, y, 1, n, 1) << '\n';
75         }
76     }
77     return 0;
78 }

```

3 數論：尤拉公式，數論分塊，Sprague-Grundy 函數，排容原理

3.1 快速冪

用法：計算 $a^b \pmod m$ 。

時間複雜度： $\mathcal{O}(\log b)$

```

1 // 計算 (a^b) % mod
2 // 競賽中 mod 通常為 1e9+7 或 998244353
3 long long fast_pow(long long a, long long b, long long mod) {
4     long long res = 1;
5     a %= mod; // 預防初始底數 a 大於等於 mod
6
7     while (b > 0) {
8         // 如果當前次方數是奇數，把當前的 a 乘進答案裡
9         if (b & 1) res = res * a % mod;
10        a = a * a % mod; // 底數平方
11        b >>= 1; // 次方數除以 2
12    }
13    return res;
14 }

```

3.2 模逆元與費馬小定理

用法：定義：若 p 為質數，且整數 a 與 p 互質，則 $a^{p-1} \equiv 1 \pmod p$ 。

邏輯應用：兩邊同除以 a 可得 $a^{p-2} \equiv a^{-1} \pmod p$ 。

這代表計算 $(x/a) \pmod p$ 等同於計算 $(x \cdot a^{p-2}) \pmod p$ 。

時間複雜度： $\mathcal{O}(\log p)$

```

1 // 模逆元 (Modular Multiplicative Inverse)
2 // 條件：mod 必須是質數 (利用費馬小定理)
3 // 應用：計算 (x / y) % mod 等同於計算 (x * inv(y, mod)) % mod
4 long long inv(long long a, long long mod) {
5     return fast_pow(a, mod - 2, mod);
6 }
7
8 // 應用範例：計算 (a / b) % mod
9 long long mod_div(long long a, long long b, long long mod) {
10    return (a % mod) * mod_inv(b, mod) % mod;
11 }

```

3.3 質數檢查

用法：單次詢問大數用 `isPrime`；若有大量詢問則建質數表。

時間複雜度： $\mathcal{O}(\sqrt{N})$

```

1 // 1. 單次質數檢驗
2 // 適用時機：只詢問極少數幾個數字，且數字可能很大 (<= 1e14)
3 bool isPrime(long long n) {
4     if (n < 2) return false;
5     if (n == 2 || n == 3) return true;
6     if (n % 2 == 0 || n % 3 == 0) return false;
7
8     // 效能優化：除了 2 和 3，質數一定出現在 6k-1 或 6k+1
9     for (long long i = 5; i * i <= n; i += 6) {
10        if (n % i == 0 || n % (i + 2) == 0) return false;
11    }
12    return true;
13 }
14
15 // =====
16
17 // 2. 埃拉托斯特尼篩法 (Sieve of Eratosthenes)
18 // 適用時機：需要大量查詢 1 ~ 10^7 內的質數
19 const int MAXN = 1e7 + 10;
20 vector<int> primes;
21 bool is_prime[MAXN];
22
23 void sieve(int n) {
24     fill(is_prime, is_prime + n + 1, true);
25     is_prime[0] = is_prime[1] = false;
26
27     for (int i = 2; i <= n; ++i) {
28         if (is_prime[i]) {
29             primes.push_back(i); // 收集質數
30
31             // 重要優化：從 i * i 開始篩，因為 i * 2, i * 3... 在之前已經被 2,
32             // 3... 篩過了
33             // 注意型態轉型避免 i * i 爆 int
34             for (long long j = (long long)i * i; j <= n; j += i) {
35                 is_prime[j] = false;
36             }
37         }
38     }

```

3.4 大數乘法

用法：未知長度的大數相乘

時間複雜度： $\mathcal{O}(N \times M)$

```

1 // 1. 大數 x 小整數 (BigInt * int)
2 // 適用場景：算階乘、連續乘上範圍在 int 內的數值
3 // 時間複雜度： $\mathcal{O}(N)$ 
4 vector<int> multiply_small(const vector<int>& a, int b) {
5     if (b == 0 || (a.size() == 1 && a[0] == 0)) return {0};
6
7     vector<int> c;
8     long long carry = 0; // 使用 long long 避免溢位
9
10    for (size_t i = 0; i < a.size() || carry > 0; i++) {
11        if (i < a.size()) carry += 1LL * a[i] * b;
12        c.push_back(carry % 10);
13        carry /= 10;
14    }
15    return c;
16 }
17
18 // 2. 大數 x 大數 (BigInt * BigInt)
19 // 適用場景：兩個長度未知的超大整數相乘
20 // 時間複雜度： $\mathcal{O}(N * M)$ 
21 vector<int> multiply_big(const vector<int>& a, const vector<int>& b) {
22     if (a.empty() || b.empty()) return {0};
23     if ((a.size() == 1 && a[0] == 0) || (b.size() == 1 && b[0] == 0)) return {0};
24
25     // 結果的最大可能長度為兩者長度相加
26     vector<int> c(a.size() + b.size(), 0);
27
28     for (size_t i = 0; i < a.size(); i++) {
29         long long carry = 0;
30         for (size_t j = 0; j < b.size() || carry > 0; j++) {
31             // 注意：要加上原本在 c[i+j] 已經算出的值
32             long long cur = c[i + j] + 1LL * a[i] * b[j] : 0)
33             + carry;
34             c[i + j] = cur % 10;
35             carry = cur / 10;
36         }
37     }
38
39     // 移除多餘的前導零 (例如 00 變成 0)
40     while (c.size() > 1 && c.back() == 0) {
41         c.pop_back();
42     }
43     return c;
44 }
45
46 // 輔助函式：將字串轉換為大數陣列 (反向存入)
47 vector<int> string_to_bigint(const string& s) {
48     vector<int> res;
49     for (int i = s.length() - 1; i >= 0; i--) {
50         res.push_back(s[i] - '0');
51     }
52     return res;
53 }
54
55 // 輔助函式：印出大數 (反向印出)
56 void print_bigint(const vector<int>& a) {
57     for (int i = a.size() - 1; i >= 0; i--) {
58         cout << a[i];
59     }
60     cout << "\n";
61 }

```

4 字串

4.1 KMP 字串匹配

用法: 尋找 **pattern** 出現位置。**pi** 陣列可求最小循環節。必須是 0-based index。

時間複雜度: $O(N + M)$

```
1 // pi 陣列 (Longest Prefix Suffix): pi[i] 代表 pattern[0..i] 的最長相同前後綴
  長度
2 vector<int> build_pi(const string& s) {
3     int m = s.length();
4     vector<int> pi(m, 0);
5     for (int i = 1, j = 0; i < m; ++i) {
6         // 如果不匹配, 就利用 pi 陣列往回跳
7         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
8         if (s[i] == s[j]) j++;
9         pi[i] = j;
10    }
11    return pi;
12 }
13
14 // 可用 pi 陣列尋找一個字串的最小循環節
15 int getMinimumPeriod(const string& s) {
16     int n = s.length();
17     if (n == 0) return 0;
18     vector<int> pi = build_pi(s);
19
20     // 取得整個字串的最長相同前後綴長度
21     int maxPrefixLength = pi[n - 1];
22
23     // 計算可能的最小循環節長度
24     int period = n - maxPrefixLength;
25
26     // 檢查總長度是否能被 period 整除
27     if (maxPrefixLength > 0 && n % period == 0) {
28         return period;
29     }
30
31     // 若無法整除, 代表沒有更小的週期, 最小循環節為字串本身
32     return n;
33 }
34 // 最終, 最小循環節長度即為回傳的 period -> s.substr(0, period);
35
36 // 執行字串匹配
37 vector<int> kmp(const string& text, const string& pattern) {
38     vector<int> res;
39     if (pattern.empty()) return res;
40
41     vector<int> pi = build_pi(pattern);
42     for (int i = 0, j = 0; i < text.length(); ++i) {
43         while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
44         if (text[i] == pattern[j]) j++;
45
46         if (j == pattern.length()) { // 完全匹配
47             res.push_back(i - j + 1); // 紀錄起始 index
48             j = pi[j - 1]; // 繼續尋找下一個可能的匹配
49         }
50     }
51     return res; // 回傳的這個 vector 包含所有完全匹配的 index 位置
52 }
```

4.2 Z-Algorithm 字串匹配

用法: **z[i]** 代表 S 與 $S[i \dots N - 1]$ 的最長共同前綴。必須是 0-based index。

時間複雜度: $O(N)$

```
1 // z[i] 代表 s[0..n-1] 與 s[i..n-1] 的最長共同前綴 (LCP) 長度
2 // 若要找 pattern 在 text 中的位置, 可傳入 s = pattern + "$" + text
3 vector<int> z_function(const string& s) {
4     int n = s.length();
5     vector<int> z(n, 0);
6     // l, r 記錄目前「最靠右」的匹配區間 [l, r]
7     for (int i = 1, l = 0, r = 0; i < n; ++i) {
8         // 如果 i 在區間內, 可以直接利用對稱性 (i - l) 抄答案
9         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
10
11         // 暴力往後擴展 (因為有前面的加速, 這裡實際上只會做很少次)
12         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
13             z[i]++;
14         }
15
16         // 如果新匹配的區間超過了原有的 r, 更新 [l, r] 視窗
17         if (i + z[i] - 1 > r) {
18             l = i;
19             r = i + z[i] - 1;
20         }
21     }
22     return z;
23 }
24
25 string pattern = "aba";
26 string text = "abacaba";
27
28 string s = pattern + "$" + text;
29
30 vector<int> z = z_function(s);
31
32 for (int i = 0; i < s.size(); ++i) {
33     if (z[i] == pattern.size()) {
34         cout << "found at " << i - pattern.size() - 1 << '\n';
35     }
36     /*
37     found at 0
38     found at 4
39     */
40 }
```

4.3 Manacher 迴文判斷

用法: 求最長迴文子字串。預處理字串塞入 # 將所有迴文轉為奇數長度。

時間複雜度: $O(N)$

```
1 // 求最長迴文
2 struct Manacher {
3     vector<int> p; // p[i] 記錄以 i 為中心的最長迴文半徑
4     string t; // 轉換後的字串
5
6     Manacher(const string& s) {
7         // 預處理: 將 "abc" 變成 "$#a#b#c#~" 來統一奇偶長度
8         // 頭尾放不同的字元 ($ 和 ~) 可省去邊界判斷
9         t = "$#";
10        for (char c : s) { t += c; t += '#'; }
11        t += '~';
12
13        int n = t.length();
14        p.assign(n, 0);
15        int c = 0, r = 0; // c: 目前最靠右的迴文中心, r: 該迴文的右界
16
17        for (int i = 1; i < n - 1; ++i) {
18            int mirror = 2 * c - i; // i 關於 c 的對稱點
19
20            // 如果 i 在迴文右界 r 內, 可以直接抄對稱點的答案
21            if (r > i) p[i] = min(r - i, p[mirror]);
22
23            // 暴力往外擴展半徑
24            while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
25                p[i]++;
26            }
27
28            // 如果這個新迴文的右界超過了 r, 更新 c 和 r
29            if (i + p[i] > r) {
30                c = i;
31                r = i + p[i];
32            }
33        }
34
35        // 回傳原字串中的最長迴文長度
36        int get_max_len() {
37            int max_len = 0;
38            for (int len : p) max_len = max(max_len, len);
39            return max_len;
40        }
41
42        // 回傳所有出現的迴文, 包含位置不同的重複字串
43        // 如果要不重複就把 vector 改成 set 進行去重, 再 for 輸出就好
44        vector<string> get_all_palindromes(const string& s) {
45            vector<string> result;
46            // 遍歷所有可能的迴文中心
47            for (int i = 1; i < t.length() - 1; ++i) {
48                // p[i] 是以 i 為中心的最大迴文長度
49                // 每次往內縮 2 (砍掉頭尾字元), 即可得到以此為中心的所有較短迴文
50                for (int len = p[i]; len > 0; len -= 2) {
51                    // 神奇的公式: (i - len) / 2 可以精準對應到原字串 s 的起點索引
52                    int start_idx = (i - len) / 2;
53                    result.push_back(s.substr(start_idx, len));
54                }
55            }
56            return result;
57        }
58
59        // 僅計算所有迴文子字串的「總數量」
60        long long count_all_palindromes() {
61            long long total = 0;
62            for (int i = 1; i < t.length() - 1; ++i) {
63                // 對於中心 i, 包含的迴文數量恰好是 ceil(p[i] / 2)
64                total += (p[i] + 1) / 2;
65            }
66            return total;
67        }
68    };
69 }
```

4.4 Trie 字典樹

用法: 字串前綴統計或最大 XOR 問題。必須是 0-based index。

時間複雜度: $O(|S|)$

```
1 struct Trie {
2     // 如果是字元 size=26, 如果是二進制 XOR size=2
3     int tree[MAXN][size] = {};
4     int passed[MAXN] = {};
5     int stop[MAXN] = {};
6     // 以上陣列的都跳過 index 0
7     int cnt = 1;
8
9     void init(){
10        for (int i=1; i<cnt; ++i){
11            for (int j=0; j<size; ++j) tree[i][j] = 0;
12            passed[i] = 0;
13            stop[i] = 0;
14        }
15        cnt = 1;
16    }
17
18    void insert(string word){
19        int curr = 1;
20        ++passed[curr]; // 經過點, ++passed
21        for (int i=0; path; i<word.length(); ++i){
22            path = word.at(i) - 'a'; // 求出一個位置的 index
23            if (tree[curr][path] == 0) tree[curr][path] = ++cnt; // 如果當前位置走
24            到目標位置不存在, 就用 ++cnt 的值
25            curr = tree[curr][path]; // 繼續往下走
26            ++passed[curr]; // 記得經過就要 ++passed
27        }
28        ++stop[curr]; // 最後停住的地方要 ++stop
29    }
30
31    int search(string word){
```

```

31     int curr = 1;
32     for(int i=0, path; i<word.length(); ++i){
33         path = word.at(i) - 'a';
34         if(tree[curr][path] == 0) return 0; // 如果從 curr 往 path 的路徑不存
           在，代表搜尋的 word 不存在
35         curr = tree[curr][path]; // 否則繼續往下走
36     }
37     return stop[curr]; // 最後回傳在 curr 位置停下 (stop) 的次數
38 }
39
40 int prefixNumber(string word){
41     int curr = 1;
42     for(int i=0, path; i<word.length(); ++i){
43         path = word.at(i) - 'a';
44         if(tree[curr][path] == 0) return 0;
45         curr = tree[curr][path];
46     }
47     return passed[curr]; // 和 search 方法一模一樣，只是回傳的對象變成 passed
48 }
49
50 void erase(string word){
51     if(search(word) == 0) return; // 如果找不到字就不管他
52     int curr = 1;
53     --passed[curr];
54     for(int i=0, path; i<word.length(); ++i){
55         path = word.at(i) - 'a';
56         if(--passed[tree[curr][path]] == 0){ // 如果 curr 往 path 的路徑上，p
           值為 1，代表後面的東西都不需要了
57             tree[curr][path] = 0; // 將 curr 不再連通 path，直接捨棄後面所有的東
           西 (因為都會被減成 p=0)
58             return;
59         }
60         curr = tree[curr][path]; // 否則繼續往下走，注意後面不需要
           再--passed[curr]，因為在 if 區塊我們已經操作完了
61     }
62     --stop[curr]; // 最後--stop[curr]
63 }
64 };

```

4.5 AC 自動機

用法：多字串匹配 (Trie + KMP)，必須是 0-based index。

時間複雜度： $\mathcal{O}(N + M)$

```

1  struct AC_Automaton {
2      static const int MAXN = 1e5 + 10;
3      int tree[MAXN][size]; // size 是字元 26，二進制 2
4      int fail[MAXN];
5      int stop[MAXN]; // 記錄有幾個字串在此結束
6      int cnt;
7
8      void init() {
9          cnt = 0;
10         new_node(); // root is 0
11     }
12
13     int new_node() {
14         fill(tree[cnt], tree[cnt] + 26, 0);
15         fail[cnt] = stop[cnt] = 0;
16         return cnt++;
17     }
18
19     // 1. 將所有關鍵字插入 Trie
20     void insert(const string& s) {
21         int curr = 0;
22         for (char c : s) {
23             int path = c - 'a';
24             if (!tree[curr][path]) {
25                 tree[curr][path] = new_node();
26             }
27             curr = tree[curr][path];
28         }
29         stop[curr]++;
30     }
31
32     // 2. 建立 Fail 指標與字典圖優化 (使用 BFS)
33     void build() {
34         queue<int> q;
35         // 將 root (0) 的所有實際存在的第一層子節點推入 queue
36         for (int i = 0; i < 26; ++i) {
37             if (tree[0][i]) {
38                 fail[tree[0][i]] = 0;
39                 q.push(tree[0][i]);
40             }
41         }
42
43         while (!q.empty()) {
44             int u = q.front();
45             q.pop();
46
47             for (int i = 0; i < 26; ++i) {
48                 if (tree[u][i]) {
49                     // 若子節點存在，它的 fail 指向父節點 fail 的對應子節點
50                     fail[tree[u][i]] = tree[fail[u]][i];
51                     q.push(tree[u][i]);
52                 } else {
53                     // 優化：若子節點不存在，直接把這條路連向 fail 的對應節點
54                     // 這樣搜尋時就絕對不會遇到死胡同，也不用往回跳
55                     tree[u][i] = tree[fail[u]][i];
56                 }
57             }
58         }
59     }
60
61     // 3. 搜尋文章，回傳所有關鍵字出現的總次數
62     int query(const string& text) {
63         int curr = 0;
64         int res = 0;
65         for (char c : text) {
66             curr = tree[curr][c - 'a']; // 感謝字典圖優化，無腦往下走即可

```

```

67         // 順著 fail 指標往上收集沿途所有匹配成功的字串 (例如配對到 "she"，
68         同時也配對到 "he")
69         int temp = curr;
70         while (temp != 0 && stop[temp] != -1) {
71             res += stop[temp];
72             stop[temp] = -1; // -1 是優化：已經算過的關鍵字不要重複算
73             temp = fail[temp];
74         }
75     }
76     return res;
77 }
78 };

```

5 圖論：著色問題，帶權二分圖最大匹配 (KM 演算法)，最大流，biconnected component 與 articulation point 找 dfn 與 low

需注意：

是否為全連通圖 (connected，只有一個 component)，對於連通圖要找 region 可以用 Euler's theorem $V-E+R=2$

undirected 無自環

directed

weighted 有權重 (點或邊)

multigraph 兩點之間有多條直接的路徑

有向圖的 outdegree, indegree，無向圖的 degree

表示圖的方式：Adjacency matrix, adjacency list

5.1 BFS 與拓樸排序

用法：無權圖最短路徑 / 拓樸排序計算 In-degree 尋找相關順序，可判斷是否有環。

時間複雜度： $\mathcal{O}(V + E)$

```

1  struct BFS_Topo {
2      int n;
3      vector<vector<int>> adj; // adjacency list，避免爆空間
4      vector<int> in_degree; // 紀錄入度，用於拓樸排序
5
6      void init(int _n) {
7          n = _n;
8          adj.assign(n + 1, vector<int>());
9          in_degree.assign(n + 1, 0);
10     }
11
12     // 有向圖，u 指向 v (u 必須在 v 之前完成)
13     void add_edge(int u, int v) {
14         adj[u].push_back(v);
15         in_degree[v]++; // v 被 u 指到，入度 +1
16     }
17
18     // 1. 拓樸排序 (Kahn's Algorithm)
19     // 回傳拓樸排序的結果。若回傳的 vector 大小小於 n，代表圖中有環
20     vector<int> topo_sort() {
21         queue<int> q;
22         vector<int> res;
23
24         // 步驟一：將所有入度為 0 (沒有前置條件) 的點加入 queue
25         for (int i = 1; i <= n; ++i) {
26             if (in_degree[i] == 0) q.push(i);
27         }
28
29         // 步驟二：開始 BFS
30         while (!q.empty()) {
31             int u = q.front();
32             q.pop();
33             res.push_back(u);
34
35             // 拔掉 u 連出去的所有邊
36             for (int v : adj[u]) {
37                 in_degree[v]--; // v 的前置條件少了一個
38                 if (in_degree[v] == 0) { // 如果前置條件都滿足了，就推入 queue
39                     q.push(v);
40                 }
41             }
42         }
43         return res; // 若 res.size() < n，說明有環，無法完成拓樸排序
44     }
45
46     // 2. 基礎 BFS：無權圖的單源最短路徑
47     vector<int> get_dist(int start) {
48         vector<int> dist(n + 1, -1);
49         queue<int> q;
50
51         dist[start] = 0;
52         q.push(start);
53
54         while (!q.empty()) {
55             int u = q.front();
56             q.pop();
57             for (int v : adj[u]) {
58                 if (dist[v] == -1) { // 沒走過
59                     dist[v] = dist[u] + 1;
60                     q.push(v);
61                 }
62             }
63         }
64         return dist;
65     }
66 };

```

5.2 DFS

用法：用遞迴走訪。

時間複雜度： $\mathcal{O}(V + E)$

```
1 struct DFS{
2     int n;
3     vector<vector<int>>> adj;
4     vector<bool> vis;
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         vis.assign(n + 1, false);
10        subtree_size.assign(n + 1, 0);
11    }
12
13    void add_edge(int u, int v) {
14        adj[u].push_back(v);
15        adj[v].push_back(u); // 無向圖就兩個都加
16    }
17
18    // 基礎 DFS：走訪與找連通塊
19    void dfs(int u) {
20        vis[u] = true;
21        // 如有需要，處理抵達 u 時的邏輯 (Pre/In/Post-order)
22
23        for (int v : adj[u]) {
24            if (!vis[v]) {
25                dfs(v);
26            }
27        }
28    }
29 };
```

5.3 匈牙利演算法

用法：無權二分圖最大匹配，透過回溯尋找增廣路徑，以達到最大匹配。
分為左 (S)、右 (T) 兩側，以及 left 陣列紀錄右側所選的對象

時間複雜度： $\mathcal{O}(V \cdot E)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 30005; // 依題目需求調整右側/左側最大點數
5 int n, m, e; // n: 左側點數, m: 右側點數
6 vector<int> G[MAXN]; // G[u] 存左側點 u 連向右側的哪些點
7 int lef[MAXN]; // lef[v] 記錄右側點 v 配對到的左側點
8 bool T[MAXN]; // T[v] 記錄右側點 v 在單次 DFS 中是否已拜訪
9
10 bool match(int u) { // 核心 DFS：幫左側點 u 尋找增廣路徑
11     for (int v : G[u]) {
12         if (T[v]) continue;
13         T[v] = true;
14
15         // 若右側點 v 還沒選對象，或其原本對象可以讓出位子
16         if (lef[v] == -1 || match(lef[v])) {
17             lef[v] = u; // 配對成功
18             return true;
19         }
20     }
21     return false;
22 }
23
24 int solve() { // 執行最大匹配
25     int ans = 0;
26     memset(lef, -1, sizeof(lef)); // 依照需求假設點編號為 0-based 或 1-based
27
28     // 假設左側點編號為 1 ~ n (若為 0 ~ n-1 請自行改迴圈範圍)
29     for (int i = 1; i <= n; ++i) {
30         memset(T, false, sizeof(T));
31         if (match(i)) {
32             ans++;
33         }
34     }
35     return ans;
36 }
37
38 int main() {
39     while (cin >> n >> m) {
40         for (int i = 0; i <= n; ++i) {
41             G[i].clear();
42         }
43
44         int u, v; // 讀取每個邊，存到 adjacency list 中
45         for (int i = 0; i < e; ++i) {
46             cin >> u >> v;
47             G[u].push_back(v);
48         }
49
50         // 依照需求做更改，這裡的範例是：找到的最大匹配數等於邊的數量，即完成最大匹配
51         if (solve() == e) cout << "YES\n";
52         else cout << "NO\n";
53     }
54     return 0;
55 }
```

5.4 Dijkstra 單源最短路徑

用法：找單個起點的最短路徑，不支援負權邊，支援自環。

時間複雜度： $\mathcal{O}((V + E) \log V)$

```
1 struct Dijkstra {
2     // 定義 pii 為 pair< 距離, 節點 >, 預設由小到大排序 (pq 用)
3     using pii = pair<long long, int>;
4     const int MAXN = 2e5 + 10;
5     const long long INF = 1e18; // 使用 1e18 避免相加時 long long 溢位
6
7     vector<pair<int, long long>> adj[MAXN]; // adj[u] = {v, weight}
```

```
8     long long dist[MAXN]; // dist[i] 為起點到 i 的距離
9     int n;
10
11     // 初始化：傳入節點數，清空圖
12     void init(int _n) {
13         n = _n;
14         for (int i = 0; i <= n; ++i) {
15             adj[i].clear();
16         }
17     }
18
19     void add_edge(int u, int v, long long w) {
20         adj[u].push_back({v, w});
21         // 若為無向圖，須加上 adj[v].push_back({u, w});
22     }
23
24     void solve(int start) {
25         fill(dist, dist + n + 1, INF); // 將距離陣列初始化為無限大
26         // pq 存起點到某一點的距離，用距離的 Min-Heap
27         priority_queue<pii, vector<pii>, greater<pii>> pq;
28
29         dist[start] = 0;
30         pq.push({0, start});
31
32         while (!pq.empty()) {
33             long long d = pq.top().first;
34             int u = pq.top().second;
35             pq.pop();
36
37             // 懶惰刪除 (Lazy Deletion)：如果取出時發現已經有更好的距離，直接丟棄
38             if (d > dist[u]) continue;
39
40             for (auto& edge : adj[u]) {
41                 int v = edge.first;
42                 long long w = edge.second;
43
44                 // 鬆弛 (Relax)
45                 if (dist[u] + w < dist[v]) {
46                     dist[v] = dist[u] + w;
47                     pq.push({dist[v], v});
48                 }
49             }
50         }
51     }
52 };
```

5.5 SPFA 單源最短路徑

用法：也是找單個起點的最短路徑，但支援負權邊。只在“可能有負邊”或“必須檢查負環”時才用

時間複雜度：預期 $\mathcal{O}(V + E)$ ，最差 $\mathcal{O}(VE)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5 const long long INF = 1e18;
6
7 struct Edge {
8     int v;
9     long long w;
10 };
11
12 vector<Edge> G[MAXN]; // adjacency list 存圖
13 long long dist[MAXN]; // 起點到任意點的距離
14 int n, m; // n 個點, m 個邊
15
16 bool SPFA(int start) { // 回傳 false 代表圖中存在負環；回傳 true 代表最短路徑計算成功
17     bool inq[MAXN]; // inq[i] 記錄點 i 目前是否在 queue 裡面
18     int cnt[MAXN]; // cnt[i] 記錄點 i 進入 queue 的總次數
19     queue<int> q;
20
21     fill(dist, dist + n + 1, INF);
22     memset(inq, false, sizeof(inq));
23     memset(cnt, 0, sizeof(cnt));
24
25     dist[start] = 0; // 從 start 節點開始
26     q.push(start);
27     inq[start] = true;
28     cnt[start] = 1;
29
30     while (!q.empty()) {
31         int u = q.front();
32         q.pop();
33         inq[u] = false; // 拿出來後就不在 queue 裡了
34
35         for (auto& edge : G[u]) {
36             int v = edge.v;
37             long long w = edge.w;
38
39             // 鬆弛操作 (relax)
40             if (dist[u] + w < dist[v]) {
41                 dist[v] = dist[u] + w;
42
43                 // 如果被鬆弛的點不在 queue 裡面，就把它加進去
44                 if (!inq[v]) {
45                     q.push(v);
46                     inq[v] = true;
47                     cnt[v]++;
48
49                     if (cnt[v] >= n) return false; // 【關鍵】一個點入隊次數 >= n (點總數)，必定有負環
50                 }
51             }
52         }
53     }
54     return true;
55 }
```

5.6 Floyd-Warshall 多源最短路徑

用法：求任意兩點之間距離。利用滾動陣列降維，**k** 必須在最外層迴圈。注意避開 INF 加上負權邊的陷阱。

時間複雜度：時間 $O(V^3)$ ，空間 $O(V^2)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 505;
5 const long long INF = 1e18;
6 long long dist[MAXN][MAXN];
7 int n, m;
8
9 void init() {
10     for (int i = 1; i <= n; ++i) {
11         for (int j = 1; j <= n; ++j) {
12             dist[i][j] = (i == j) ? 0 : INF;
13         }
14     }
15 }
16
17 // 注意：讀邊時若為多重邊 (Multigraph, 即 A->B 不只一條 direct path)，需寫成
18 dist[u][v] = min(dist[u][v], w);
19
20 void floyd() {
21     // 中繼點 k 必須在最外層！
22     for (int k = 1; k <= n; ++k) {
23         for (int i = 1; i <= n; ++i) {
24             for (int j = 1; j <= n; ++j) {
25                 // 必須確認 i->k 和 k->j 都是通的
26                 if (dist[i][k] < INF && dist[k][j] < INF) {
27                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
28                 }
29             }
30         }
31     }
32 }
```

6 幾何

6.1 點與向量基礎 (Point & Vector)

用法：幾何基礎，定義 struct 與 operator。包含浮點數誤差處理 sign，以及核心工具 dot (內積) 與 cross (外積)。

時間複雜度： $O(1)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-9; // 用來處理浮點數誤差的極小值
5
6 // 符號判斷函數 (極度重要!)
7 // 判斷一個浮點數是 正數 (1)、負數 (-1) 還是 0(0)
8 int sign(double x) {
9     if (fabs(x) < EPS) return 0;
10     return x < 0 ? -1 : 1;
11 }
12
13 struct Point {
14     double x, y; //通用 double
15     Point(double _x = 0, double _y = 0) : x(_x), y(_y) {}
16
17     // 向量加減法與常數乘除
18     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
19     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
20     Point operator*(double d) const { return Point(x * d, y * d); }
21     Point operator/(double d) const { return Point(x / d, y / d); }
22
23     // 判斷兩點是否相等 (利用 sign 來避免浮點數誤差)
24     bool operator==(const Point& p) const {
25         return sign(x - p.x) == 0 && sign(y - p.y) == 0;
26     }
27
28     bool operator<(const Point& a, const Point& b) const {
29         if (a.x != b.x) return a.x < b.x;
30         return a.y < b.y;
31     }
32 };
33
34 // 在幾何中，向量和點的表示法一樣，給它一個別名增加可讀性
35 using Vector = Point;
36
37 // 1. 內積 (Dot Product)
38 // 計算方式：x1*x2 + y1*y2
39 // 實戰意義：用來判斷兩向量夾角。大於 0 為銳角，小於 0 為鈍角，等於 0 為垂直。
40 double dot(Vector a, Vector b) {
41     return a.x * b.x + a.y * b.y;
42 }
43
44 // 2. 外積 (Cross Product)
45 // 計算方式：x1*y2 - x2*y1
46 // 實戰意義：用來判斷方向與面積。
47 // 若 cross(a, b) > 0，代表 b 在 a 的逆時針方向 (偏左)
48 // 若 cross(a, b) < 0，代表 b 在 a 的順時針方向 (偏右)
49 // 若 cross(a, b) == 0，代表 a 和 b 平行或共線
50 double cross(Vector a, Vector b) {
51     return a.x * b.y - a.y * b.x;
52 }
```

6.2 判斷線段相交 (Segment Intersection)

用法：利用外積進行跨立實驗。避開斜率除以零的問題，可處理端點相切與共線重疊。

時間複雜度： $O(1)$

```
1 #include <algorithm>
2 using namespace std;
3
4 // 依賴前一節的 Point 結構與 sign, cross 函數
5
6 // 輔助函數：判斷點 P 是否在線段 AB 的「邊界框 (Bounding Box)」內
7 // 實戰意義：當三點共線時，單靠外積無法確認 P 是否在線段上 (可能在延長線上)
8 bool on_segment(Point p, Point a, Point b) {
9     return p.x >= min(a.x, b.x) && p.x <= max(a.x, b.x) &&
10         p.y >= min(a.y, b.y) && p.y <= max(a.y, b.y);
11 }
12
13 // 核心函數：判斷線段 AB 與線段 CD 是否相交
14 bool segment_intersect(Point a, Point b, Point c, Point d) {
15     // 1. 計算四個外積值 (跨立實驗)
16     // 意義：判斷 C 點和 D 點分別在直線 AB 的哪一側
17     double c1 = cross(b - a, c - a);
18     double c2 = cross(b - a, d - a);
19
20     // 意義：判斷 A 點和 B 點分別在直線 CD 的哪一側
21     double c3 = cross(d - c, a - c);
22     double c4 = cross(d - c, b - c);
23
24     // 2. 規範相交 (Strictly Intersect)
25     // 如果 C, D 在 AB 兩側 (外積正負號相反，相乘 < 0)
26     // 且 A, B 在 CD 兩側，則保證兩線段相交
27     if (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0) {
28         return true;
29     }
30
31     // 3. 特殊情況處理 (端點相交 或 共線重疊)
32     // 如果外積為 0，代表該點在另一條直線上。
33     // 接著必須用 on_segment 確保它真的落在「線段區間」內，而不是在延長線上。
34     if (sign(c1) == 0 && on_segment(c, a, b)) return true;
35     if (sign(c2) == 0 && on_segment(d, a, b)) return true;
36     if (sign(c3) == 0 && on_segment(a, c, d)) return true;
37     if (sign(c4) == 0 && on_segment(b, c, d)) return true;
38
39     return false; // 以上皆非，則不相交
40 }
```

6.3 點與線段關係 (Point & Line)

用法：判斷 1. 點是否在線段上 2. 求點到直線/線段最短距離。利用內外積避免解方程式與浮點數除以零。

時間複雜度： $O(1)$

```
1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point, Vector, sign, cross, dot
5
6 // 輔助函數：求向量長度
7 // 原理：向量自己跟自己做內積，等於長度的平方
8 double length(Vector a) {
9     return sqrt(dot(a, a));
10 }
11
12 // 1. 判斷點 P 是否在線段 AB 上
13 // 原理：
14 // (1) cross(p-a, b-a) == 0 代表三點共線
15 // (2) dot(a-p, b-p) <= 0 代表 PA 向量與 PB 向量方向相反 (即 P 夾在 A, B 中間)
16 bool on_segment(Point p, Point a, Point b) {
17     return sign(cross(p - a, b - a)) == 0 && sign(dot(a - p, b - p)) <= 0;
18 }
19
20 // 2. 點 P 到「直線」AB 的距離
21 // 原理：平行四邊形面積 = 底 * 高 => 高 = 面積 / 底
22 double dist_to_line(Point p, Point a, Point b) {
23     // 面積剛好是外積的絕對值，底邊長是 AB 的長度
24     return fabs(cross(b - a, p - a)) / length(b - a);
25 }
26
27 // 3. 點 P 到「線段」AB 的最短距離 (賽場極易錯考點)
28 // 原理：用內積判斷 P 點的「投影」是否掉出線段外
29 double dist_to_segment(Point p, Point a, Point b) {
30     if (a == b) return length(p - a); // A, B 根本是同一個點
31
32     // 鈍角測試 1：角 PAB 是鈍角，代表 P 在 A 的外側，離 A 最近
33     if (sign(dot(b - a, p - a)) < 0) return length(p - a);
34
35     // 鈍角測試 2：角 PBA 是鈍角，代表 P 在 B 的外側，離 B 最近
36     if (sign(dot(a - b, p - b)) < 0) return length(p - b);
37
38     // 如果都不是鈍角，代表 P 的投影乖乖落在線段 AB 上，距離就是點到直線距離
39     return dist_to_line(p, a, b);
40 }
```

6.4 任意多邊形面積

用法：利用外積求多邊形面積 (鞋帶公式)。頂點必須依順時針或逆時針順序給定。支援凹凸多邊形。

時間複雜度： $O(N)$

```
1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXV = 1005;
7 Point poly[MAXV]; // 儲存多邊形的頂點 (必須依照順時針或逆時針順序排列)
8 int n; // 頂點數量
9
10 // 求任意多邊形面積 (Shoelace Formula)
11 double polygon_area() {
12     double area = 0;
13     for (int i = 0; i < n; ++i) {
14         // 讓最後一個點與第 0 個點連線，形成封閉圖形
15         int next = (i + 1) % n;
```

```

16     area += cross(poly[i], poly[next]);
17 }
18 // 外積和有可能是負的（取決於順逆時針），所以取絕對值並除以 2
19 return fabs(area) / 2.0;
20 }
21
22 /*
23 補充：假設兩個矩形的左下角與右上角座標分別為
24 矩形 A：左下 (x1, y1)，右上 (x2, y2)
25 矩形 B：左下 (x3, y3)，右上 (x4, y4)
26
27 把矩形投影到 X 軸上，就變成了兩個線段 [x1, x2] 與 [x3, x4]
28 它們重疊的長度，等於「兩個右邊界的較小值」減去「兩個左邊界的較大值」
29 如果算出來是負數，代表它們根本沒碰到，直接跟 0 取最大值即可
30
31 因此計算矩形交集面積即為
32 int W = max(0, min(x2, x4) - max(x1, x3));
33 int H = max(0, min(y2, y4) - max(y1, y3));
34 int intersect_area = W * H;
35 */

```

6.5 凸包 Convex Hull

用法：求包圍所有點的最小凸多邊形。先排 X 再排 Y 。由左至右建下半部，由右至左建上半部。利用外積判斷並踢除右轉點。

```

時間複雜度： $\mathcal{O}(N \log N)$ 
1 #include <algorithm>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXN = 100005;
7 Point pts[MAXN]; // 存放所有輸入的點
8 Point hull[MAXN]; // 存放最終構成凸包的點
9 int n; // 輸入點的數量
10
11 // 回傳凸包上的頂點數量
12 int convex_hull() {
13     // 點太少直接就是凸包
14     if (n <= 2) {
15         for (int i = 0; i < n; ++i) hull[i] = pts[i];
16         return n;
17     }
18
19     // 1. 將所有點由左至右、下至上排序
20     sort(pts, pts + n);
21
22     int m = 0; // m 代表目前凸包 (hull) 裡面有幾個點
23
24     // 2. 蓋下半部凸包 (Lower Hull) - 從左走到右
25     for (int i = 0; i < n; ++i) {
26         // 【核心邏輯】
27         // cross <= 0 代表「往右轉」或是「直走」。
28         // 凸包的邊緣必須一直「往左轉」，如果發現往右轉（凹進去了），就把最後一個點
29         // 踢掉！
30         // （若想保留共線的點，把 <= 改成 < 即可）
31         while (m >= 2 && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
32             m--; // 踢掉最後一個點
33         }
34         hull[m++] = pts[i]; // 把新點加進來
35     }
36
37     // 3. 蓋上半部凸包 (Upper Hull) - 從右走回左
38     // 注意：下半部已經蓋了 m 個點，為了不踢掉下半部的點，我們設定底線為 t
39     int t = m + 1;
40     for (int i = n - 2; i >= 0; --i) {
41         // 邏輯跟上面一模一樣，只是倒著走
42         while (m >= t && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
43             m--;
44         }
45         hull[m++] = pts[i];
46     }
47
48     // 4. 起點（最左下的點）會在頭尾被重複加入一次，所以總數量要減 1
49     return m - 1;
50     // 如果需要知道所有選擇的點，就是 hull[0]-hull[m-1]
51     // 然後有了這些點又可以用面積公式直接得到整個的面積
52 }

```

7 樹論

7.1 Disjoint Set

用法：解決集合、快速合併的問題。優化必須路徑壓縮，可選小掛大

```

時間複雜度： $\mathcal{O}(1)$ 
1 struct DSU {
2     vector<int> f, sz; // 避免使用 size 撞名
3
4     void init(n) {
5         f.resize(n);
6         sz.assign(n, 1); // 將每個獨立集合的大小初始化為 1
7         for (int i = 0; i < n; ++i) f[i] = i;
8     }
9
10    int find(int x) { // 路徑壓縮
11        return f[x] == x ? x : (f[x] = find(f[x]));
12    }
13
14    bool isSameSet(int a, int b) {
15        return find(a) == find(b);
16    }
17
18    // 回傳 bool 有助於在 Kruskal 演算法中判斷是否成功加上一條邊，否則 void 即可
19    bool unite(int a, int b) {

```

```

20        int fa = find(a), fb = find(b);
21        if (fa == fb) return false;
22
23        // 小掛大優化：永遠讓 fa 是比較大的集合
24        if (sz[fa] < sz[fb]) swap(fa, fb);
25        sz[fa] += sz[fb];
26        f[fb] = fa;
27        return true;
28    }
29 };

```

7.2 Kruskal 最小生成樹

用法：需搭配並查集使用。將所有邊排序後，利用併查集檢查是否形成環。

```

時間複雜度： $\mathcal{O}(E \log E)$ 
1 // 讓 Edge 支援比較大小，排序時會自動依據權重 w 由小到大排
2 struct Edge {
3     int u, v;
4     long long w;
5     bool operator<(const Edge& rhs) const {
6         return w < rhs.w;
7     }
8 };
9
10 struct Kruskal {
11     int n;
12     vector<Edge> edges;
13
14     void init(int _n) {
15         n = _n;
16         edges.clear();
17     }
18
19     void add_edge(int u, int v, long long w) {
20         edges.push_back({u, v, w});
21     }
22
23     // 回傳最小生成樹的總權重。若圖不連通則回傳 -1
24     long long solve() {
25         sort(edges.begin(), edges.end()); // 1. 將所有邊依權重排序
26
27         DSU dsu;
28         dsu.init(n + 1); // 2. 呼叫併查集（預設 1-based index）
29
30         long long total_weight = 0;
31         int edge_cnt = 0; // 記錄成功連接了幾條邊
32
33         for (auto& e : edges) {
34             // 3. unite 右回傳 true，代表 u 和 v 原本不相連，合併成功（無環）
35             if (dsu.unite(e.u, e.v)) {
36                 total_weight += e.w;
37                 edge_cnt++;
38
39                 // V 個點只要 V-1 條邊即完全連通，提早結束優化
40                 if (edge_cnt == n - 1) break;
41             }
42         }
43
44         // 檢查是否有成功把所有點連通
45         return (edge_cnt == n - 1) ? total_weight : -1;
46     }
47 };
48
49 /*
50 延伸閱讀
51 找第二小的 Minimum Spanning Tree
52 改成 Maximum Spanning Tree
53 對 Min ST 做 DFS
54 */

```

8 DP: LIS, LDS, 分錢幣問題, Kadane's algo 找最大子序列問題

DP 通用解題四步驟：

1. 定義陣列意義：明確寫出 $dp[i]$ (或 $dp[i][j]$) 代表什麼。
2. 找出遞迴關係式：推導出當前狀態如何由過去狀態轉移而來。
3. 設定邊界條件：初始化起點或基礎 dp 值（如： $dp[0] = 0$ ）。
4. 確認遍歷順序：決定迴圈是從前到後（Bottom-Up）還是後到前（Top-Down）計算。（通常是 Bottom-Up，思考「我是如何到這一步的」去推）

8.1 01 Knapsack 背包

用法：解決物品不可拆分的情況，若可拆分用 Greedy 拿比例最佳的部分

```

時間複雜度： $\mathcal{O}(N)$ 
1 /*
2 題目敘述：給定 N 個物品的重量 Wi 和價值 Vi，和個容量為 W 的背包。選取若干件物品
3 放入背包，在不超過背包容量的情況下，背包內物品價值總和最大為何？
4
5 定義 dp[i][j] 表示：在考慮前 i 個物品，且考慮重量 <= j 的情況下，所能得到的最大價值
6 是多少（假設 index 0 不放東西）
7 初始：對於所有的 i=0, dp[i][j] = 0（不考慮任何物品的情況下，最大價值是 0），也可以
8 直接預設所有位置都是 0
9 轉移函式：dp[i][j] = max(dp[i-1][j], dp[i-1][j-Wi] + Vi);
10 -> 兩種情況：
11 1. 不放第 i 件物品，最大價值與前一項相同

```

```

9 2. 放入第 i 件物品，最大價值為 扣除目標重量後的最大價值 加上當前物品的價值之總和
10
11 適用情況：需要 backtrack
12 -> 從 dp[0][0] 開始檢查：
13 如果 dp[i][j] == dp[i-1][j] 表示沒有拿第 i 個，接著讓 i = i-1 繼續跑
14 否則我們有拿第 i 個，接著跳到 dp[i-1][j-w[i]]
15 重複直到 i=0 (沒東西了) 或 j=0 (重量用完了)
16 /*
17
18 int dp[0][0] = 0; // 初始化為 0
19 for (int i = 1; i <= MAXN; ++i) // 從考慮第一個物品到第 N 個物品
20 {
21     for (int j = 0; j < w[i]; ++j) // 如果當前考慮重量小於 Wi，代表沒辦法裝下第
22         i 個物品，直接繼承前一項
23         dp[i][j] = dp[i-1][j];
24     for (int j = w[i]; j <= MAXW; ++j) // 否則就用 transition function 找 max
25         dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]);
26 }
27 cout << dp[0][0] << '\n'; // 最終答案在最右下那格
28
29 /*
30 ===== 滾動陣列優化 =====
31 概念：在計算考慮第 i 個物品時，我們只會用到第 i-1 行的資料，可以看成陣列中的上一
32 筆舊資料
33 透過想法上的極致壓縮，覆蓋無用資料空間，可將空間複雜度降至 O(W)
34 限制是 j 必須由大往小跑，如果由小至大會重複使用到陣列資料導致答案錯誤
35 /*
36 int dp[0][0] = 0;
37 for (int i = 1; i <= MAXN; ++i) // 注意一樣是要跑 MAXN 次，我們只是把陣列大小減
38     少了
39     {
40         for (int j = MAXW; j >= w[i]; --j) // dp[j] 在等號左邊表示當前，等號右邊表
41             示上一筆資料
42             dp[j] = max(dp[j], dp[j-w[i]] + v[i]); // 不需要考慮 j < w[i] 的情況，
43             因為它會繼承上一個結果
44     }
45     cout << dp[0][0] << '\n';
46
47 /*
48 ===== 恰好裝滿問題 / 子集和 =====
49 題目敘述：給定 N 樣物品，每件物品的重量分別為 Wi，問可不可以將物品分成等重的兩堆。
50
51 題解：首先判斷總重是否為二的倍數，如果不是就直接輸出 NO 然後下一個 case
52 如果可以的話就使用一維 DP 解，總重/2 當作 MAXW (dp 大小)，每件行李的重量是價值，
53 看是否存在 dp[MAXW] = MAXW (在裝入一半行李的情況下，總重剛好也是一半)
54
55 題目：N 個物品 (同一項商品可被多個人同時取用)，G 個人各自有容量為 Wi 的背包，求所
56 有人能拿的最大價值總和。
57
58 題解：每個人挑選互相獨立，不需重複計算。(查表)
59 1. 以所有商品跑一次標準一維 0/1 背包，每次讀入 Wi 與 Vi 時就 run 一次，建立 dp
60     陣列。
61 2. 結束後跑一遍 dp[i] = max(dp[i], dp[i-1]) 確保單調性。
62 3. 讀入每個人的容量 W，總答案 ans += dp[W]。
63 */

```

8.2 LCS 最長共同子序列 (Sequence)

用法：二維 DP 與 Backtrack 找原始子序列

時間複雜度： $O(NM)$

```

1 // LCS (線性 DP) : dp[i][j] 代表的是 s1 的前 i 個字元，與 s2 的前 j 個字元。這
2   是一個從索引 0 開始的前綴
3
4 struct LCS {
5     // 求解 LCS 並回傳最長共同子序列的字串
6     // 若只需長度，直接回傳 dp[n][m] 即可
7     string solve(const string& s1, const string& s2) {
8         int n = s1.length(), m = s2.length();
9
10        // dp[i][j] 代表 s1 的前 i 個字元與 s2 的前 j 個字元的 LCS 長度
11        // 宣告 n+1 與 m+1 是為了讓 index 0 作為「空字串」的邊界條件 (全為 0)
12        vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
13
14        // 1. 狀態轉移 (填表)
15        for (int i = 1; i <= n; ++i) {
16            for (int j = 1; j <= m; ++j) {
17                if (s1[i-1] == s2[j-1]) {
18                    // 若字元相同，繼承左上角的答案 + 1
19                    dp[i][j] = dp[i-1][j-1] + 1;
20                } else {
21                    // 若字元不同，取上方或左方的最大值
22                    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
23                }
24            }
25        }
26
27        // 2. 回溯 (Backtracking) 找回原本的字串
28        string res = "";
29        int i = n, j = m;
30        while (i > 0 && j > 0) {
31            if (s1[i-1] == s2[j-1]) {
32                // 如果字元相同，這一定是 LCS 的一部分
33                res += s1[i-1];
34                i--; j--; // 往左上角回退
35            } else if (dp[i-1][j] > dp[i][j-1]) {
36                // 如果不同，往數值比較大的方向走 (此處為上方)
37                i--;
38            } else {
39                // 否則往左方走
40                j--;
41            }
42        }
43        reverse(res.begin(), res.end());
44        return res;
45    }
46 };

```

```

42 // 因為是從尾巴推回去的，最後要把字串反轉
43 reverse(res.begin(), res.end());
44 return res;
45 }
46 };
47
48 /*
49 補充：Longest Palindrome Sequence 最長迴文子序列
50 給定一個字串，問你該字串的最長迴文子序列 "長度" 為多少
51 做法就是對 s1 與 s2=reverse(s1) 做一次 LCS，即可得到 "最長長度"
52
53 變化 2：求最少插入/刪除次數以形成迴文
54 如果只求長度 -> strlen - LPS(s)
55 */

```

8.3 區間 DP 概念 (最長迴文子序列)

用法：區間 DP 是由字串的內部向外擴張，利用已經算好的較短字子串，來推導出較長字子串的答案。

時間複雜度： $O(N^2)$

```

1 // LPS (區間 DP) : dp[i][j] 代表的是同一個字串 s 裡面，從索引 i 到 j 的這段閉區
2   間 (也就是子字串 s[i...j])
3
4 struct IntervalDP {
5     // 求解最長迴文子序列 (LPS) 長度
6     int solveLPS(const string& s) {
7         int n = s.length();
8         if (n == 0) return 0;
9
10        // dp[i][j] 代表字串 s 區間 [i, j] 內的最長迴文子序列長度
11        vector<vector<int>> dp(n, vector<int>(n, 0));
12
13        // 1. 狀態轉移 (填表)
14        // 注意 i 是從字串的「尾巴」往「頭」掃
15        for (int i = n-1; i >= 0; --i) {
16            // Base case: 單一字元本身就是長度為 1 的迴文
17            dp[i][i] = 1;
18
19            // j 從 i 的下一個位置開始向右擴張
20            for (int j = i+1; j < n; ++j) {
21                if (s[i] == s[j]) {
22                    // 若頭尾字元相同，長度為內部區間的最長迴文長度 + 2
23                    // 因為 i 是倒著掃，j 是正著掃，所以 dp[i+1][j-1] 絕對已經
24                    // 算過了
25                    dp[i][j] = dp[i+1][j-1] + 2;
26                } else {
27                    // 若頭尾不同，看是捨棄左邊 (s[i]) 還是捨棄右邊 (s[j]) 比較
28                    // 長
29                    dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
30                }
31            }
32        }
33
34        // 最終答案就是涵蓋整個字串的區間 [0, n-1]
35        return dp[0][n-1];
36    }
37 };
38
39 /*
40 有關迴文題目的補充
41 變化 1：計算迴文子序列的「總數」 (排容原理)
42 令 dp[i][j] 表示「A 從索引 i 開始的後綴」與「B 從索引 j 開始的前綴」所能構成的最
43 長條件迴文長度。
44 如果 s[i] == s[j]，則 dp[i][j] = dp[i+1][j-1] + 1
45 如果 s[i] != s[j]，則 dp[i][j] = max(dp[i+1][j], dp[i][j-1])
46 回溯過程 (可使用 deque)：
47 如果 s[i] == s[j]：兩端直接放 s[i]，然後 i++, j--
48 如果 s[i] != s[j]：代表是選擇在右邊插入與 s[i] 相同的字元。兩端放
49 s[i]，然後 j--
50 如果 dp[i+1][j] > dp[i][j-1]：代表是選擇在左邊插入與 s[j] 相同的字元。兩端放
51 s[j]，然後 i--
52 如果 dp[i+1][j] == dp[i][j-1]：代表補 s[i] 或補 s[j] 都能達到最少次數。這時為了
53 字典序最小，我們直接比較 min(s[i], s[j])，誰小就補誰！
54 注意：當 i==j，記得把 s[i] 放到答案正中間
55
56 變化 3：跨字串 (左 A 右 B) 的迴文
57 令 dp[i][j] 表示「A 從索引 i 開始的後綴」與「B 從索引 j 開始的前綴」所能構成的最
58 長條件迴文長度。
59 如果 A[i] == B[j]，則 dp[i][j] = dp[i+1][j-1] + 2
60 如果 A[i] != B[j]，則 dp[i][j] = max(dp[i+1][j], dp[i][j-1])
61 */

```

8.4 跳格子

用法：給定一組陣列，固定跳躍距離，問你最少需要跳幾次，以及所跳過的格子 index。如果數值達到 long long，INF 需設為 1e18 或 0x3f3f3f3f3f3f3f3f

時間複雜度： $O(N \times K)$

```

1 //
2 題目出處：PUPC 2025 PC River Crossing
3 題目敘述：給定一組陣列，1 表示可達，0 表示不可達，問你最少需要跳幾次可以跳到終點，
4 以及跳時經過的路徑
5 如果不需要輸出路徑，可以直接用 greedy，但要輸出路徑就需要用 DP
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 int main() {
11     int kase;
12     cin >> kase;
13 }

```

```

13 while(kase--){
14     int n;
15     cin >> n;
16     vector<int> v(n), ans(n, 0x3f3f3f3f), pre(n, -1);
17 // v 存原始陣列, ans 存抵達該格的最短步數, pre 存 在最短步數的情況下, 我上一格走
    的位置
18     for(int i=0; i<n; ++i) cin >> v[i]; // 讀入陣列
19     ans[0] = 0; // 初始化: 跳到第 0 格的距離是 0
20     for(int i=1; i<n; ++i){ // 接下來每次遍例我的前三格
21         if(v[i] == 0) continue; // 如果該格不可走,
22         for(int j=1; j<=3; ++j){ // 這裡的 3 是題目設定的, 請改成該題目設置的數值
23             if(i-j >= 0 && v[i-j] == 1 && ans[i-j] != 0x3f3f3f3f){
24 // 判斷: 1. 在邊界內, 2. 該格可走, 3. 重複確認該格可走 (如不可走, ans[i-j] 會
                直接 skip, 保留 INF)
25                 if(ans[i] >= ans[i-j] + 1){ // 注意題目要求: 相同距離的情況下, 格子要
                    選哪一個, 在這題中紀錄的格子要越小越好
26                     ans[i] = ans[i-j] + 1;
27                     pre[i] = i-j;
28                 }
29             }
30         }
31     }
32     if(ans[n-1] == 0x3f3f3f3f) cout << -1 << '\n';
33     else{
34         cout << ans[n-1] << '\n';
35
36         int i = n-1;
37         stack<int> st;
38         st.push(n-1);
39         while(i > 0){
40             st.push(pre[i]);
41             i = pre[i];
42         }
43
44         bool first = true;
45         while(!st.empty()){
46             if(!first) cout << ' ';
47             first = false;
48             cout << st.top();
49             st.pop();
50         }
51         cout << '\n';
52     }
53 }
54 return 0;
55 }

```

9 Greedy: 二分搜

擴充一：區間合併 (Merge Intervals) 情境：給你一堆區間，把所有重疊的部分合併起來 (Leetcode 56)。貪心策略：依照「開始時間」排序。如果下一個區間的開始時間 $\leq$ 當前區間的結束時間，則合併 (更新當前結束時間為兩者最大值)；否則將當前區間放入答案，換下一個。

擴充二：多點排程 / 優先佇列貪心 (Huffman 概念) 情境：經典的「切木板問題」或「合併果子 (碎紙機)」，每次合併兩堆，成本是兩堆的重量和，求最小總成本。貪心策略：這裡不需要排序和掃描線，而是直接把所有元素丟進 Min-Heap (Priority Queue)。每次 pop 出最小的兩個相加，把總和加進成本，再把總和 push 回 Heap，直到 Heap 只剩下一個元素。

9.1 一維區間全覆蓋

用法：需要先把輸入轉化成一個包含 l, r 的 struct，自訂 operator<，起始位置由小到大排序

時間複雜度： $\mathcal{O}(N \log N)$

```

1 /*
2 題目敘述：給定 N 個灑水器，每個灑水器包含座標 P 與半徑 R，已知草坪長度為 L，寬度
    為 W，請問最少需要開啟幾個灑水器就能覆蓋整個區間 (或指定哪些灑水器)
3
4 題解：先將灑水器轉化為一個 struct point，用 l, r 區間表示能覆蓋的左右範圍，然
    後自訂 operator<
5 1. 用開始位置排序
6 2. 每次都在 小於當前右界的區間中，選擇能往右最多的那個
7 3. 重複直到右區間 <L，得到最少數量；或者是在當前右邊界內找不到可選擇的表示不存在
    解
8 */
9 const double eps = 1e-8;
10 struct point{
11     double L, R;
12     bool operator<(const point&o) const{
13         return L < o.L;
14     }
15 };
16
17 vector<point> v;
18 double pos, rad;
19 for(int i=0; i<n; ++i){
20     cin >> pos >> rad;
21     double real;
22     if(rad - w/2.0 < eps) continue; // 判斷如果半徑小於寬度就不可能被選，直接
        不加入
23     real = sqrt(rad*rad - (w/2.0)*(w/2.0)); // real 表示實際可覆蓋距離
24     v.push_back({pos-real, pos+real});
25 }
26 sort(v.begin(), v.end());
27
28 int cnt = 0; // 總計數，如果要指定哪些灑水器就用 vector 存每次加入的
29 int i = 0;
30 double furthest = 0.0;
31 bool noAns = false;
32
33 while(L - furthest > eps){ // L > furthest
34     double cur = furthest;
35     while(i < v.size() && v[i].L - furthest < eps){

```

```

36         cur = max(cur, v[i].R);
37         ++i;
38     }
39     if(cur - furthest < eps){
40         noAns = true;
41         break;
42     }
43     furthest = cur;
44     ++cnt;
45 }
46 if(noAns) cout << -1 << '\n';
47 else cout << cnt << '\n';

```

9.2 間格跳躍

用法：給定一組陣列，距離為 index 的值，問你最少需要跳多少次才能到終點

時間複雜度： $\mathcal{O}(N)$

```

1 int jump(vector<int>& nums) {
2     int jumps = 0; // 總跳躍次數
3     int current_end = 0; // 「當前這一步」能到達的最遠邊界
4     int farthest = 0; // 探索過程中，發現「下一步」能到達的最遠距離
5     bool noAns = false;
6
7     // 注意：迴圈只走到 nums.size() - 2
8     // 因為如果已經站在最後一個位置，就不需要再起跳了
9     for (int i = 0; i < nums.size() - 1; ++i) {
10         // 邊走邊看，更新下一跳的最遠潛力
11         farthest = max(farthest, i + nums[i]);
12
13         // 當走到了「當前這步」的極限邊界時，就必須結算，進行「下一跳」
14         if (i == current_end) {
15             if (current_end == farthest){ // 如果被迫跳但發現一步都沒跳就代表走
                不到終點
16                 noAns = true;
17                 break;
18             }
19             jumps++;
20             current_end = farthest; // 將邊界擴展到剛才探索到的最遠距離
21
22             if (current_end >= nums.size() - 1) break; // 提前結束的優化
23         }
24     }
25     if (noAns) return -1;
26     return jumps;
27 }

```

9.3 最大不重疊區間

用法：對一組任務做排程，請問最多可以移除多少個任務 -> 最早結束的先選。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 int countRemoves(vector<pair<int, int>> &intervals) {
2     if(intervals.empty()) return 0; // 不需要移除任何東西
3
4     // 用結束時間做排序
5     sort(intervals.begin(), intervals.end(), [](const auto&a, const
        auto&b){return a.second < b.second;});
6
7     int removed = 0;
8     int cur_end = intervals[0].second; // 一開始必定選第一個任務
9     for(int i=1; i<intervals.size(); ++i){ // 遍歷所有任務
10         if(cur_end > intervals[i].first) ++removed; // 如果我目前的 end 能覆蓋
            到當前 i 的左邊界就移除 i
11         else cur_end = intervals[i].second; // 否則我就要選 i，並設為當前右邊
            界
12     }
13
14     return removed;
15 }

```

9.4 最小覆蓋區間

用法：給定一組陣列，包含開始時間與結束時間，求最少需要幾個房間就能完成所有任務

時間複雜度： $\mathcal{O}(N \log N)$

```

1 struct Interval {
2     int st, ed;
3     bool operator<(const Interval &o) const {
4         if (st != o.st) return st < o.st; // 先以開始時間排序
5         return ed < o.ed; // 開始時間相同，較早結束的排前面
6     }
7 };
8
9 int minMeetingRooms(vector<Interval>& intervals) {
10     if (intervals.empty()) return 0;
11
12     // 1. 依照開始時間排序
13     sort(intervals.begin(), intervals.end());
14
15     // 2. 建立一個 Min-Heap，專門用來儲存「使用中會議室的結束時間」
16     // !! 如果需要印出順序，只要把 int(結束時間) 改成 pair<int, int>(結束時間,
        房間編號)!!
17     priority_queue<int, vector<int>, greater<int>> min_heap;
18
19     // 3. 掃描每一個會議
20     for (const auto& interval : intervals) {
21         // 如果 pq 不為空，且最早結束的會議已經可以使用了 (當前會議的開始時間 >=
            最早空出來的房間時間)
22         if (!min_heap.empty() && interval.st >= min_heap.top()) {
23             min_heap.pop(); // 該會議室空出來了，把舊的結束時間 pop 掉 (準備重
                複利用)
24         }
25     }

```

```
26         // 把當前會議的結束時間塞進去 (代表佔用了一間會議室)
27         // 不管是開了一間新房間，還是接續前一間，當前的結束時間都要記錄
28         min_heap.push(interval.ed);
29     }
30
31     return min_heap.size(); // 最終 Heap 裡面剩下幾個元素，就代表我們同時需要幾
    間會議室
32 }
```

10 MISC: Random Select(QuickSort), 雙指針, 滑動窗口, C++ algorithm 庫

10.1 模板

用法: 競賽用基礎模板

時間複雜度: $\mathcal{O}(HowFastYouType)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 int main(){
6     ios::sync_with_stdio(0); cin.tie(0); // 或使用 scanf
7     return 0;
8 }
```